On-Device AI 실습
LLM Pruning & Speculative Decoding

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

실습 목차

1. **LLM Pruning 실습**
   - Magnitude-based Pruning & Wanda Pruning 구현 실습
2. **Vanilla Speculative Decoding**
   - Qwen2.5-3B/Qwen2.5-0.5B 활용 Vanilla SD 구현 실습

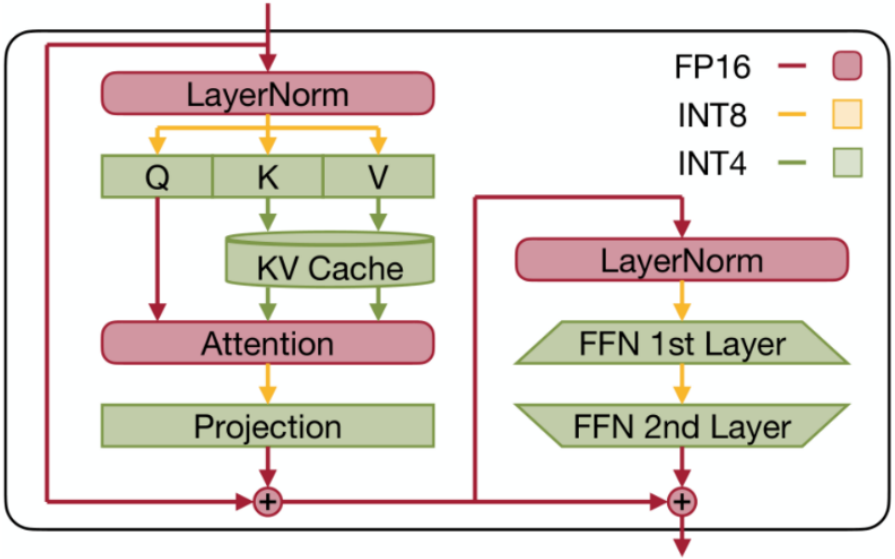
3. **EAGLE-style tree (어텐션 마스크 / 확장 / 검증)**
   - Tree mask, Tree verify, Accept/reject 실습
4. **DFlash Parallel Block Drafting [ICML '26]**
   - Diffusion LLM을 통한 한번의 forward로 전체 drafting 실습

PART 3

Pruning for LLM

Post-Training Pruning for LLMs

- **When Pruning LLMs, the targets are the projection layer of the Multi-head attention block and the Fully Connected (FC) layers of the Feed-Forward Network(FFN)**



Perplexity

```
# Negative log likelihood 계산
nlls = []
for i in tqdm.tqdm(range(nsamples), desc="evaluating..."):
    # 배치 데이터 준비
    batch = testenc[:, (i * 2048):((i + 1) * 2048)].to(model.device)

    # 모델 추론
    with torch.no_grad():
        lm_logits = model(batch).logits

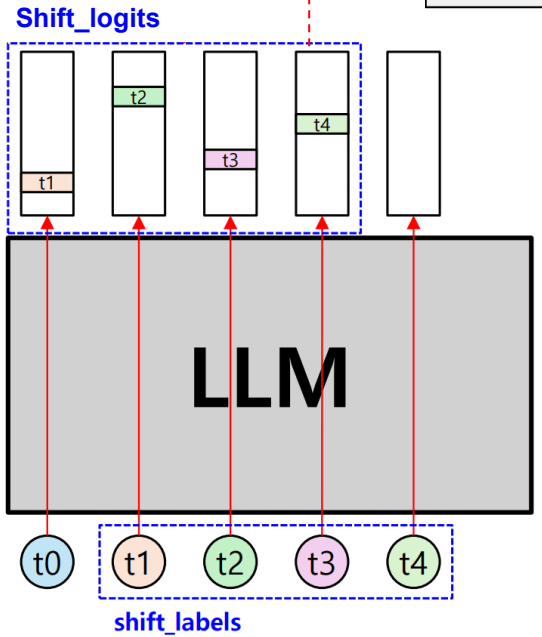
    # 로짓과 레이블 시프트
    shift_logits = lm_logits[:, :-1, :].contiguous().float()
    shift_labels = testenc[:, (i * 2048):((i + 1) * 2048)][:, 1:]

    # 손실 계산
    loss_fct = nn.CrossEntropyLoss()
    loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
    neg_log_likelihood = loss.float() * 2048
    nlls.append(neg_log_likelihood)

# Perplexity 계산 및 반환
return torch.exp(torch.stack(nlls).sum() / (nsamples * 2048))
```

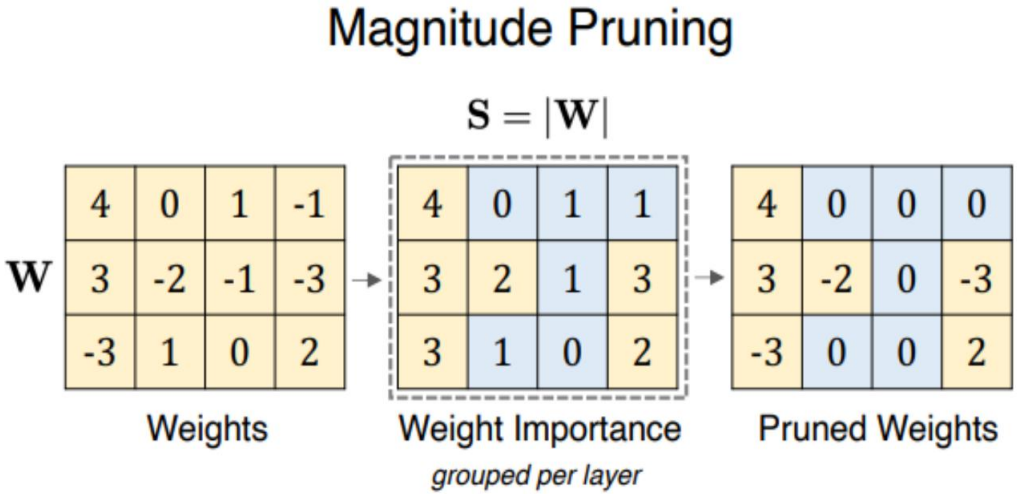
CrossEntropyLoss

$$\exp\left(\frac{-\log P_1 - \log P_2 - \log P_3 - \log P_4}{4}\right) = e^{\frac{\log \frac{1}{P_1 P_2 P_3 P_4}}{4}} = \sqrt[4]{\frac{1}{P_1 P_2 P_3 P_4}}$$



[실습 1] Magnitude-based Pruning 구현

- **Pruning is performed using only the magnitude of the weights**



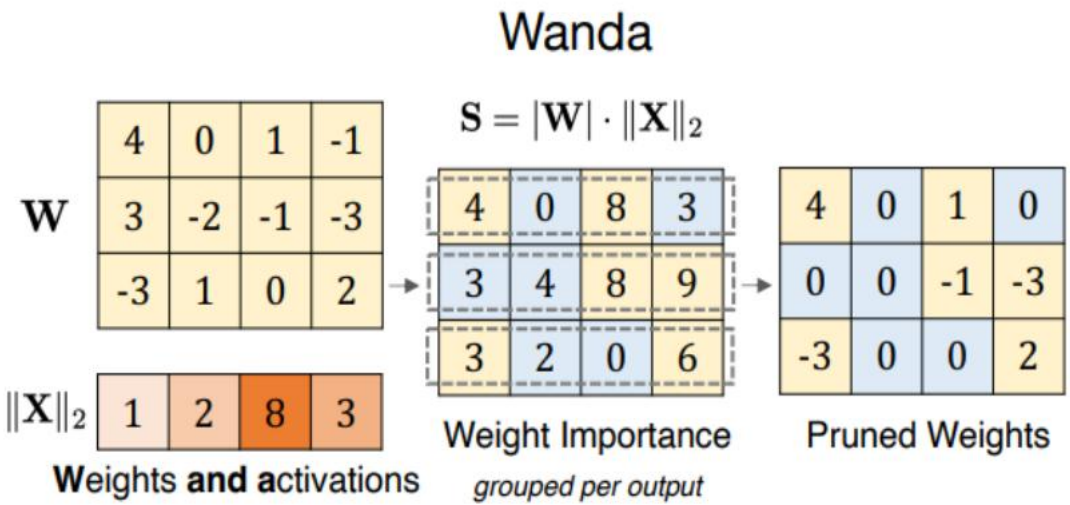
[실습 1] Answer


```
###################### YOUR CODE STARTS HERE ######################
num_elements = W.numel()
num_zeros = round(num_elements * sparsity)
importance = torch.abs(W)
threshold = torch.kthvalue(importance.flatten(), num_zeros)[0]
mask = importance > threshold
###################### YOUR CODE ENDS HERE ########################
```

[실습 2, 3] Wanda Pruning 구현

- **Feature Values are sampled through calibration**

$$\|\mathbf{X}\|_2 = \sqrt{\sum_i \mathbf{X}_i^2}$$



[실습 2, 3] Answer


```
###################### YOUR CODE STARTS HERE ######################
# activation_norm을 계산하세요.
# x.shape => (hidden_size, batch_size)
activation_norm = torch.norm(x, p=2, dim=1) ** 2
# activation_norm.shape => (hidden_size)
###################### YOUR CODE ENDS HERE ########################
```

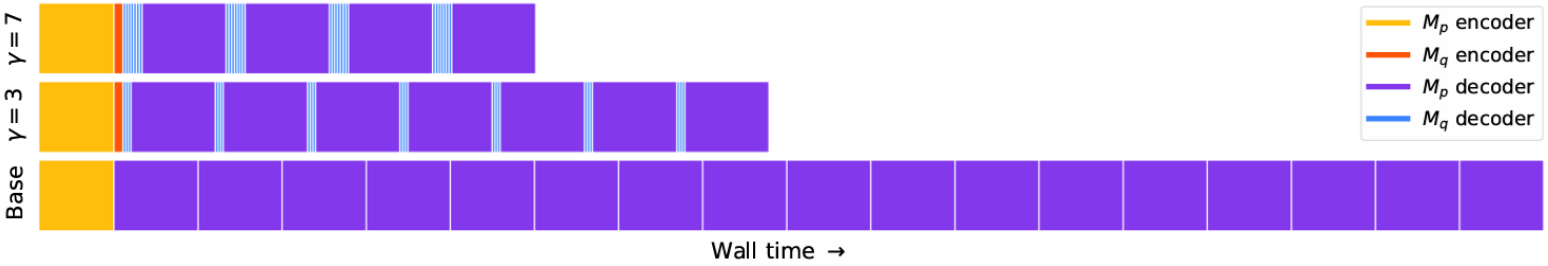
```
###################### YOUR CODE STARTS HERE ######################
row, col = W.shape
num_zeros_per_row = round(col * sparsity)
importance = torch.abs(W) * input_feat[n]
threshold = torch.kthvalue(importance, num_zeros_per_row, dim=1)[0]
mask = importance > threshold.reshape(row, 1)
###################### YOUR CODE ENDS HERE ########################
```

PART 3

Speculative Decoding 기초

값싼 모델이 제안하고, target이 한 번에 검증한다

Speculative Decoding: Draft and Verify



- **decoding이 memory-bound이므로, 한 토큰을 만드는 값으로 여러 토큰을 검증할 수 있다**
  - Draft — 작은 모델이 다음 γ개 토큰을 추측한다
  - Verify — target 모델이 γ+1개 위치를 한 번의 forward로 채점한다
  - Accept — 앞에서부터 일치하는 prefix를 수락하고, target 자신의 다음 토큰 1개를 더 붙인다

Why It Is Lossless

- **Greedy decoding**
  - A drafted token is accepted only if it equals the target's own argmax
  - The first mismatch is replaced by the target's token
  - The output is token-for-token identical to plain autoregressive decoding
- **Sampling**
  - Rejection sampling preserves the target distribution exactly
- ***SD는 다른 압축 기법과 함께 사용 가능***

Quantization / Pruning

- 모델 자체를 바꾼다 → lossy
- 정확도 하락을 감수한다

Speculative Decoding

- 모델을 바꾸지 않는다 → lossless
- 속도만 얻고 품질은 그대로

Speedup 모델

$$\text{speedup} \approx \tau / (1 + c \cdot \gamma)$$

τ acceptance length	**c** drafter cost ratio	**γ** draft length
라운드당 평균 수락 토큰 수	target 대비 drafter 1회 forward 비용	라운드당 draft 토큰 수

- **τ만 키우면 되는 것이 아님 → drafting 비용이 분모에 있다**
  - draft를 길게 뽑으면 τ는 오르지만 c·γ도 함께 오른다
  - 이후 나오는 모든 기법은 이 trade-off에 대한 서로 다른 답이다

[실습 4] Vanilla Speculative Decoding 구현

3-2. Draft Generation

```
def draft_tokens(model, x, cache, gamma):
    tokens = []
    # greedy_step()으로 draft token을 gamma개 순차 생성
    for _ in range(_____):
        x, cache = ____________________
        tokens.append(x)
    return torch.cat(tokens, dim=-1), cache
```

- **draft는 순차 (gamma 회 forward)**
  - 여기가 SD의 비용 항 c·γ

3-3. Target Verification

```
def verify_draft(target, x, drafted, cache):
    # 마지막 committed token + draft token을 하나의 sequence로
    verify_ids = ____________________
    result = target(verify_ids, past_key_values=cache,
                    use_cache=True)
    preds = result.logits[0].argmax(-1)
    return preds, result.past_key_values
```

- **verify는 단 1회 forward**
  - γ+1개 위치를 동시에 채점
  - 마지막 확정 토큰을 앞에 붙이는 이유
  - preds[0]이 d1과 비교할 대상이 되기 때문

[실습 4] Vanilla Speculative Decoding 구현

3-4. Accept / Reject

```
def accept_draft(drafted, preds, gamma):
    n = 0
    for i in range(gamma):                    # 앞에서부터 비교
        if ________:
            n += 1
        else:
            break                             # 첫 mismatch에서 정지

    # accept된 token 뒤에 target의 prediction 하나를 추가
    new_tokens = torch.cat([drafted[:, :n], preds[n].view(1, 1)], dim=1)
    return new_tokens, n
```

- **Draft와 Target prediction을 앞에서부터 비교**
- **첫 mismatch에서 정지**

[실습 4] Answer


```python
# ========================================================================
# 3-2. Draft Generation
# ========================================================================

@torch.inference_mode()
def draft_tokens(model, x, cache, gamma):
  tokens = []

  ########################## 실습 시작 ##########################

  # TODO: 다음 draft token을 생성하세요.
  # HINT: greedy_step()을 사용해 draft token을 gamma개 순차적으로 생성하세요.
  for _ in range(gamma):
    x, cache = greedy_step(model, x, cache)
    tokens.append(x)

  ########################## 실습 끝 ##########################

  return torch.cat(tokens, dim=-1), cache
```

```python
@torch.inference_mode()
def verify_draft(target, x, drafted, cache):

  ########################## 실습 시작 ##########################

  # TODO : 마지막 committed token과 draft token들을 하나의 sequence로 연결하세요.
  # HINT: torch.cat()을 사용해 마지막 committed token과 draft token들을 sequence 방향으로 연결하세요.
  verify_ids = torch.cat([x, drafted], dim=-1)

  result = target(
    verify_ids,
    past_key_values=cache,
    use_cache=True
  )

  preds = result.logits[0].argmax(-1)

  ########################## 실습 끝 ##########################

  return preds, result.past_key_values
```

[실습 4] Answer


```python
def accept_draft(drafted, preds, gamma):

  ########################## 실습 시작 ##########################

  #TODO: Draft와 Target prediction을 앞에서부터 비교하고 첫 mismatch에서 멈추세요.
  n = 0
  for i in range(gamma):
    if drafted[0, i] == preds[i]:
      n += 1
    else:
      break

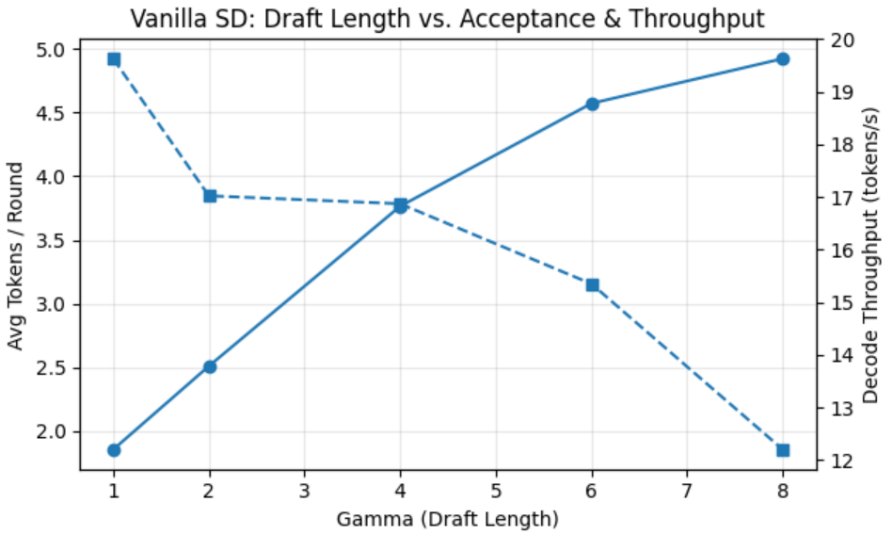
  # HINT: accept된 draft token 뒤에 Target의 prediction 하나를 추가하세요.
  new_tokens = torch.cat([drafted[:, :n],preds[n].view(1, 1)],dim=1)

  ########################## 실습 끝 ##########################

  return new_tokens, n
```

Gamma Sweep


```
Gamma= 1 | Avg tokens/round=1.86 | TPS=19.6
Gamma= 2 | Avg tokens/round=2.51 | TPS=17.0
Gamma= 4 | Avg tokens/round=3.76 | TPS=16.9
Gamma= 6 | Avg tokens/round=4.57 | TPS=15.4
Gamma= 8 | Avg tokens/round=4.92 | TPS=12.2
```



- **γ를 키우면 τ는 계속 오름**
  - 한 라운드에서 더 멀리 전진할 수 있기 때문
- **TPS는 어느 지점에서 꺾임**
  - drafter forward도 γ번 늘어나기 때문 (분모의 c·γ)
  - 틀린 뒤의 draft는 전부 버려지므로 낭비가 커진다
- **하드웨어의 특성과 테스크의 특성에 맞춰서 draft length 결정 필수**

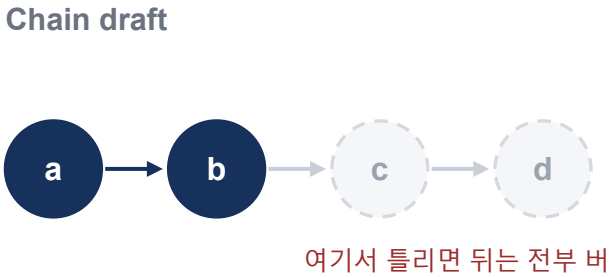
PART 4

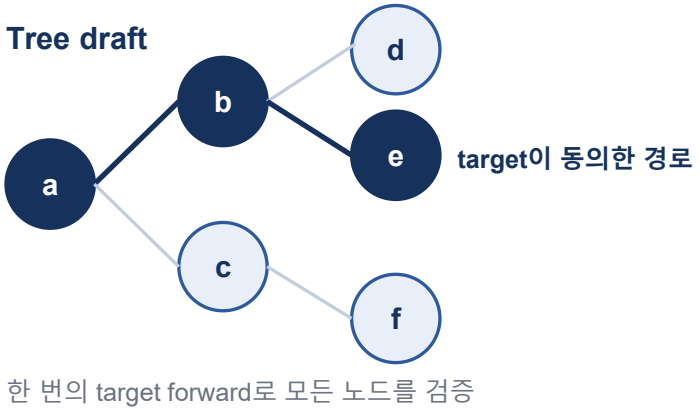
EAGLE 계열: Tree Drafting

하나의 사슬 대신 여러 후보 경로를 동시에 검증한다

From Chain to Tree

- **A chain draft bets everything on one path**
  - 중간에 한 번 틀리면 그 뒤 토큰은 전부 폐기
- **A tree draft hedges**
  - 여러 후보 경로를 같은 target forward에서 함께 검증
  - target이 동의하는 가지를 고름





The EAGLE Family

	drafting 대상	트리	특징
EAGLE-1	feature (hidden state) 공간	정적 트리	token 대신 feature를 예측하고, shifted token으로 불확실성을 보정
EAGLE-2	feature	**동적 트리**	drafter의 confidence에 따라 문맥마다 다른 트리를 만든다
EAGLE-3	다층 fused feature → token 직접 예측	**동적 트리**	feature 예측 제약을 제거, 학습 데이터 스케일링에 잘 반응

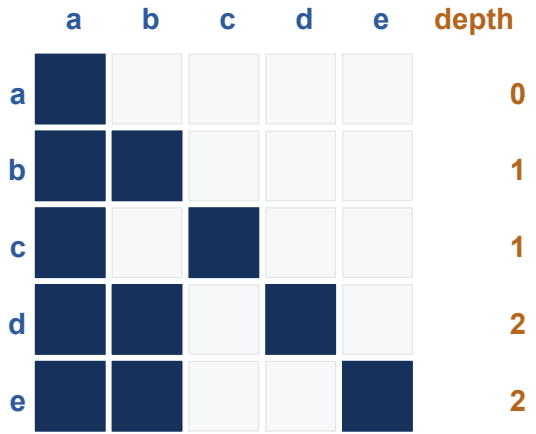
본 실습은 AngelSlim/Qwen3-4B_eagle3 체크포인트를 사용하고, 트리 구성·마스크·검증 로직을 직접 구현한다.

Tree Attention Mask and Position IDs

- **Flatten the tree into one sequence, then restrict what each node can see**
  - ancestor-only mask: prefix + 자기 조상 + 자기 자신
  - 형제·사촌 노드는 서로에게 보이면 안 된다

- **Positions follow tree DEPTH, not sequence order**
  - position_id(node) = prefix_len - 1 + depth(node)
  - 형제 노드는 같은 position id를 공유한다

flatten: root a → b, c → d, e



행 = query 노드, 열 = 볼 수 있는 노드
(b와 c는 서로 보이지 않는다)

[실습 5-A] Tree Mask & Position IDs

Answer

```
def build_tree_mask_and_positions(mask_index_list, total_tokens,
device=None):
    # 자기 자신은 항상 볼 수 있음 + 0번(anchor)은 모두가 볼 수 있음
    tree_mask = torch.eye(total_tokens + 1, dtype=torch.bool,
device=device)
    tree_mask[:, 0] = True

    # child는 parent가 보는 node를 그대로 상속한다
    for child, parent in enumerate(mask_index_list, start=1):
        ____________________________________

    # 보이는 node 수 = 트리 depth
    tree_position_ids = tree_mask.sum(dim=1).long() - 1
    tree_mask = tree_mask.float()[None, None]
    return tree_mask, tree_position_ids
```

- **부모가 자식보다 먼저 오도록 정렬되어 있음**
  - 그래서 한 번의 순회로 조상 집합이 완성된다
- **|= 는 OR 결과를 왼쪽에 저장**
  - 조상의 가시 집합을 그대로 상속받는 연산
- **depth를 따로 계산할 필요가 없다**
  - 보이는 노드 수가 곧 깊이

[실습 5-A] Answer


```python
def build_tree_mask_and_positions(
  mask_index_list,
  total_tokens,
  device=None
):
  # 자기 자신은 항상 볼 수 있음
  tree_mask = torch.eye(
    total_tokens + 1,
    dtype=torch.bool,
    device=device
  )
  tree_mask[:, 0] = True

  ######################### 실습 시작 ########################

  # ToDo: child가 parent가 볼 수 있는 node들을 그대로 볼 수 있게 하세요.
  # |= 는 OR 결과를 왼쪽에 저장하는 연산입니다.
  for child, parent in enumerate(mask_index_list, start=1):
    tree_mask[child] |= tree_mask[parent]

  ######################### 실습 끝 ########################

  # 보이는 node 수를 이용해 Tree depth 계산
  tree_position_ids = tree_mask.sum(dim=1).long() - 1
  tree_mask = tree_mask.float()[None, None]
  return tree_mask, tree_position_ids

# 구현을 실제 EAGLE에 연결
student_tree.build_tree_mask_and_positions = build_tree_mask_and_positions
```

```
tensor([[1, 0, 0, 0],
        [1, 1, 0, 0],
        [1, 0, 1, 0],
        [1, 1, 0, 1]], dtype=torch.int32)

Position IDs: [0, 1, 1, 2]
```

[실습 5-B] Tree Verification & Acceptance

4-4. Tree Verification

```
######################### 실습 시작 #########################

# To Do: Tree candidate 전체를 Target model에 한 번에 입력하세요.
outputs, tree_logits, hidden_state = model( _________ )

######################### 실습 끝 #########################
```

4-5. Tree Acceptance

```
######################### 실습 시작 #########################

# To Do: 가장 길게 일치한 branch와 그 accept length를 선택하세요.
best_candidate = _________________
accept_length = ___________________

######################### 실습 끝 #########################
```

[실습 5-B] Answer


```python
# ================================================================
# 4-4. [실습 2] Tree Verification
# ================================================================

import eagle.model.ea_model as ea_module

def tree_decoding(
  model,
  tree_candidates,
  past_key_values,
  tree_position_ids,
  input_ids,
  retrieve_indices,
):
  position_ids = tree_position_ids + input_ids.shape[1]
  if position_ids.dim() == 1:
    position_ids = position_ids.unsqueeze(0)


  ######################### 실습 시작 ########################

  # To Do: Tree candidate 전체를 Target model에 한 번에 입력하세요.
  outputs, tree_logits, hidden_state = model(
    tree_candidates,
    output_orig=True,
    past_key_values=past_key_values,
    position_ids=position_ids,
  )

  ######################### 실습 끝 ##########################
  return logits, hidden_state, outputs
```

```python
# ================================================================
# 4-5. [실습 3] Tree Acceptance
# ================================================================

def evaluate_posterior(
  logits,
  candidates,
  logits_processor,
):

  # 본 실습에서는 Greedy Decoding만 사용
  assert logits_processor is None

  # 각 branch에서 Draft와 Target prediction 비교
  matches = (
    candidates[:, 1:].to(logits.device)
    == logits[:, :-1].argmax(dim=-1)
  ).int()

  # 첫 mismatch 전까지 연속으로 맞은 token 수
  accept_lengths = torch.cumprod(
    matches,
    dim=1
  ).sum(dim=1)

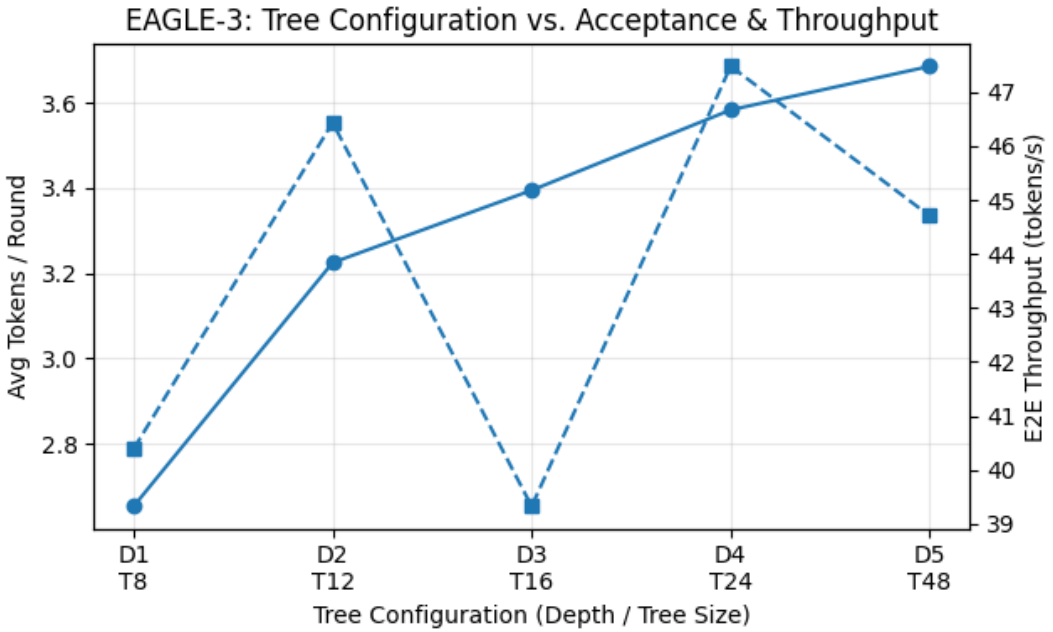

  ######################### 실습 시작 ########################

  # To Do: 가장 길게 일치한 branch와 그 accept length를 선택하세요.
  best_candidate = accept_lengths.argmax()
  accept_length = accept_lengths[best_candidate]

  ######################### 실습 끝 ##########################
  return best_candidate, accept_length, sample_p
```

[실습 5-A,B] 결과


```
Depth=1 | Tree Size= 8 | Avg Tokens/Round=2.65 | TPS=40.4
Depth=2 | Tree Size=12 | Avg Tokens/Round=3.23 | TPS=46.4
Depth=3 | Tree Size=16 | Avg Tokens/Round=3.39 | TPS=39.3
Depth=4 | Tree Size=24 | Avg Tokens/Round=3.58 | TPS=47.5
Depth=5 | Tree Size=48 | Avg Tokens/Round=3.69 | TPS=44.7
```




```
Generated Tokens : 129
Target Rounds    : 35
Avg Tokens/Round : 3.69
E2E TPS          : 26.5 tok/s

[Output]
Sure! Here's a Python function that reverses a **linked list**

### Step 1: Define the Node class

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

Step 2: Define the function to reverse the linked list

```python
def reverse_linked_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next
```
```

Depth × Tree Size: Drafting은 여전히 Sequential

- **트리를 키우면 τ는 계속 올라간다**
  - 후보 경로가 많아질수록 target이 동의할 확률이 높다
- **speedup은 그만큼 따라오지 않는다**
  - depth를 1 늘릴 때마다 drafter forward가 한 번씩 더 필요하다
  - EAGLE의 drafting은 여전히 autoregressive
  - 이 sequential 비용이 속도 식의 분모를 키운다

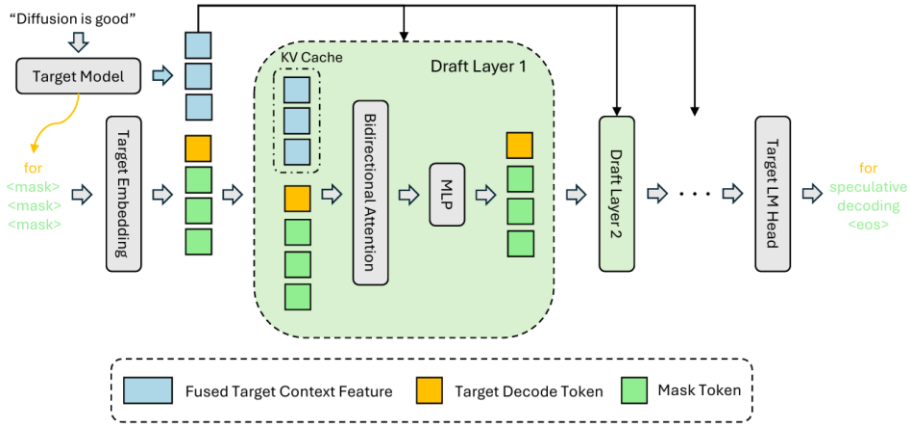


PART 5

DFlash: Parallel Block Drafting

drafting의 sequential 비용 자체를 없앤다 · ICML 2026

DFlash: Block Diffusion Drafting



- **autoregressive drafter를 block diffusion drafter로 교체한다**
  - 미래 블록을 [MASK] 토큰으로 초기화한다
  - 한 번의 denoising forward로 블록 전체의 분포를 동시에 예측
  - target hidden state를 context feature로 drafter KV에 주입
  - drafter의 logits은 target의 LM head를 통과시킨다

- **Why a diffusion LM works here**
  - 단독 생성 품질은 AR보다 약함
  - verification이 품질을 보장하므로 필요한 것은 병렬성뿐

- **Reported: 6x+ lossless speedup, up to 2.5x over EAGLE-3**

한 라운드 비교: Chain / Tree / Block

Vanilla SD

drafter forward × γ



Target forward × 1 (verify)

EAGLE

drafter forward × depth



Target forward × 1 (verify)

DFlash

drafter forward × 1



블록 전체를 동시에

Target forward × 1 (verify)

세 방식 모두 target 검증은 라운드당 1회. 차이는 drafting에 드는 sequential forward 수

[실습 6] Parallel Block Drafting

5-1. Parallel Block Drafting

```
######################### 실습 시작 ########################

 # To Do: 마지막 num_draft개의 hidden state만 선택하세요.
 draft_hidden = _____________

 # To Do: model.compute_logits()을 사용해 모든 위치의 vocabulary logits을 계산하세요.
 draft_logits = _____________

 # To Do: torch.argmax()를 사용해 모든 위치의 token을 동시에 선택하세요.
 draft_tokens = ___________________

 ######################### 실습 끝 ########################
```

- **argmax에 loop가 없음**
  - 블록 내 모든 위치의 토큰이 같은 forward에서 결정된다
  - compute_logits는 target의 LM head를 사용한다
  - block size를 키워도 drafter forward 수는 그대로 1회

[실습 6] Answer


```
    ######################### 실습 시작 ########################

    # To Do: 마지막 num_draft개의 hidden state만 선택하세요.
    draft_hidden = draft_hidden[:, -num_draft:, :]

    # To Do: model.compute_logits()을 사용해 모든 위치의 vocabulary logits을 계산하세요.
    draft_logits = model.compute_logits(draft_hidden, output_head)

    # To Do: torch.argmax()를 사용해 모든 위치의 token을 동시에 선택하세요.
    draft_tokens = torch.argmax(draft_logits, dim=-1)

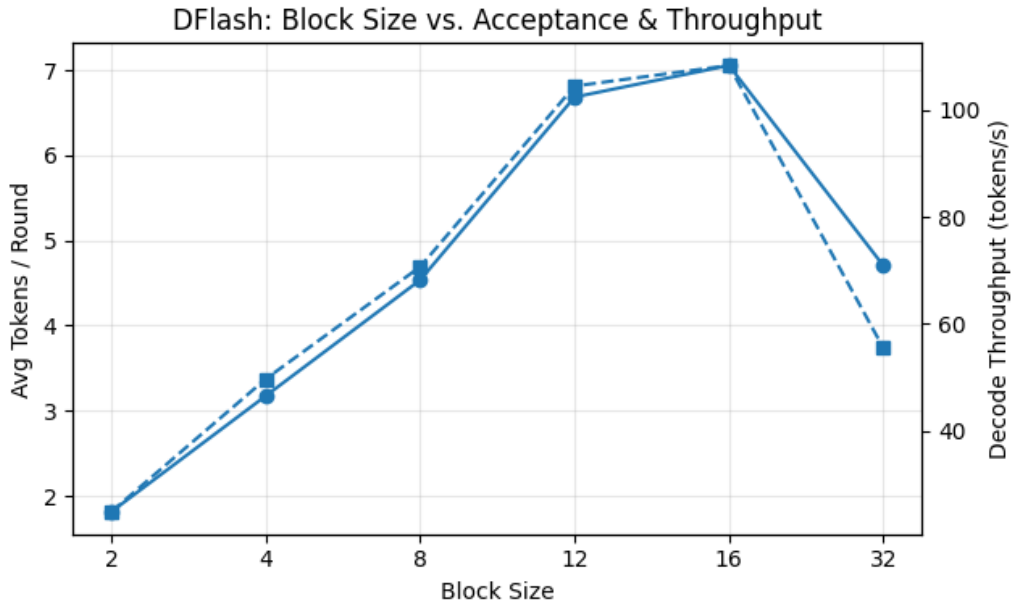
    ######################### 실습 끝 ##########################
```

```
Draft Shape : torch.Size([1, 15])

[Parallel Draft]
['!', ' Here', "'s", ' a', ' Python', ' function', ' that', ' revers', 'es', ' a', ' linked', ' list', ' list', '###', "'ll"]
```

[실습 6] 결과


```
Block Size= 2 | Avg Tokens/Round=1.81 | Decode TPS=24.9
Block Size= 4 | Avg Tokens/Round=3.17 | Decode TPS=49.7
Block Size= 8 | Avg Tokens/Round=4.54 | Decode TPS=70.7
Block Size=12 | Avg Tokens/Round=6.68 | Decode TPS=104.5
Block Size=16 | Avg Tokens/Round=7.06 | Decode TPS=108.4
Block Size=32 | Avg Tokens/Round=4.70 | Decode TPS=55.5
```




```
Block Size       : 16
Generated Tokens : 128
Target Rounds    : 18
Avg Tokens/Round : 7.06
Decode TPS       : 108.9 tok/s

[Output]
Sure! Here's a Python function that reverses a **linked list**.

### Step 1: Define the Node class

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

Step 2: Define the function to reverse the linked list

```python
def reverse_linked_list(head):
    prev = None
    current = head

    while current:
        next node = current
```
```